

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

Duration :60 Mins

On class

Total Marks:50

QUESTIONS:

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform string processing, function calls, array manipulation, conditional execution, pointer operations, switch-case selection, and loop execution. However, the program contains several errors introduced during different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler";

    int count;

    count = findLength();

    if(count > 5)
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    printf("%d\n", marks[4]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2
            printf("Option 2\n");
            break;

        default:
            printf("Invalid Choice\n");
    }
```

```

}

int *ptr;
printf("%d\n", *ptr);

int total = 100;
int students = 0;

float average = total / students;

int i = 1;

while(i <= 5)
{
    printf("%d\n", i);
}

return 0;
}

```

Tasks

- Identify all errors in the program. Mention the line number, error type, and reason for each error. (5 Marks)
- Classify the identified errors as Lexical, Syntax, Semantic, Logical, or Runtime errors in a suitable table. (5 Marks)
- Explain at which phase of compilation each category of error is detected. Also mention which errors are detected only during execution. (4 Marks)
- Rewrite the program after correcting all the errors so that it executes successfully. (4 Marks)
- Suggest any three compiler features or warnings that can help detect such programming errors before execution. (2 Marks)

Q2. The following C program is intended to perform function calls, array processing, conditional execution, switch-case selection, and loop operations. However, several syntax errors have been intentionally introduced by violating the grammatical rules of the C language. These errors include missing delimiters, unmatched parentheses, incorrect control structures, malformed function definitions, and incomplete statements. Such errors are normally detected during the syntax analysis (parsing) phase of a compiler.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```

int maximum(int x, int y)
{
    if(x > y)
        return x;
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n");
            break;

        case 90:
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    } while(max > 75);

    if(max == 75)
        printf("Limit Reached\n");
}

return 0;

```

Tasks

(a) Identify all syntax errors in the program. Mention the line number and reason for each error. (4 Marks)

(b) Classify the identified syntax errors under the appropriate categories and present them in a table. (3 Marks)

(c) Correct the syntax errors and rewrite the complete program. (4 Marks)

(d) Explain how the parser detects syntax errors and how error recovery enables further parsing. (2 Marks)

(e) Suggest any two improvements that can help a compiler generate better syntax error messages. (2 Marks)

Q3. The following C program is intended to perform factorial computation, function calls, array processing, pointer manipulation, conditional execution, switch-case selection, and loop operations. However, the program contains several errors introduced during different phases of compilation and execution, including lexical, syntax, semantic, intermediate code generation, code optimization, and runtime errors.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```
int factorial(int n)
```

```
{
```

```
    if(n <= 1)
```

```
        return 1;
```

```
    return n * factorial(n - 1);
```

```
}
```

```
int main()
```

```
{
```

```
    int a num = 5;
```

```
    int value = 10
```

```
    int result;
```

```
    result = factorial(j);
```

```

if(result == 100)
{
    printf("Large Number\n");
}

switch(result)
{
    case 24:
        printf("Factorial of 4\n");
        break;

    case 120:
        printf("Factorial of 5\n");
        break;

    default:
        printf("Other Value\n");
}

int numbers[5] = {2,4,6,8,10};
printf("%d\n", numbers[5]);

int *ptr;
printf("%d\n", *ptr);

int a = 50;
int b = 0;
int c = a / b;

int sum = result + 0 + value;

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
}

return 0;
}

```


Q1. (a) Identify all errors in the program

Line No.	Error	Error Type	Reason
10	char name[20] = "Compiler"	Syntax Error	Missing semicolon (;) at the end of the statement.
12	count = findLength();	Semantic Error	findLength() requires one argument (name), but none is passed.
13	if(count > 5	Syntax Error	Missing closing parenthesis) in the if condition.
19	printf("%d\n", marks[4]);	Runtime Error	Array index out of bounds. Valid indices are 0–3.
25	case 2	Syntax Error	Missing colon (:) after case 2.
33	printf("%d\n", *ptr);	Runtime Error	Pointer ptr is uninitialized and dereferenced.
36	float average = total / students;	Runtime Error	Division by zero since students = 0.
38–41	while(i <= 5)	Logical Error	i is never incremented, causing an infinite loop.

(b) Classification of Errors

Error	Category
Missing semicolon after char name[20] = "Compiler"	Syntax Error
Missing argument in findLength()	Semantic Error
Missing) in if(count > 5	Syntax Error
Accessing marks[4]	Runtime Error
Missing : after case 2	Syntax Error
Dereferencing uninitialized pointer	Runtime Error
Division by zero	Runtime Error
Infinite loop (i not incremented)	Logical Error

Lexical Errors: None in the given program.

(c) Compilation Phases and Error Detection

Error Category	Detected During	Explanation
Lexical Error	Lexical Analysis	Invalid tokens or illegal characters are detected.

Syntax Error	Syntax Analysis (Parser)	Checks grammar rules such as missing semicolons, parentheses, or colons.
Semantic Error	Semantic Analysis	Checks meaning of the program, function arguments, variable declarations, and types.
Logical Error	Not detected by compiler	Program compiles but produces incorrect behavior (e.g., infinite loop).
Runtime Error	Execution Time	Occurs only when the program runs (e.g., division by zero, invalid memory access).

Detected only during execution:

- Array index out of bounds
- Uninitialized pointer dereference
- Division by zero
- Infinite loop

(d) Corrected Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler";
    int count;

    count = findLength(name);

    if(count > 5)
    {
        printf("Length is greater than 5\n");
    }
}
```



```
int marks[4] = {85, 90, 75, 80};  
printf("%d\n", marks[3]);
```

```
int choice = 2;
```

```
switch(choice)  
{  
    case 1:  
        printf("Option 1\n");  
        break;  
  
    case 2:  
        printf("Option 2\n");  
        break;  
  
    default:  
        printf("Invalid Choice\n");  
}
```

```
int value = 10;  
int *ptr = &value;  
printf("%d\n", *ptr);
```

```
int total = 100;  
int students = 5;  
float average = (float)total / students;  
printf("Average = %.2f\n", average);
```

```
int i = 1;  
while(i <= 5)  
{  
    printf("%d\n", i);  
    i++;  
}
```

```
return 0;
}
```

(e) Three Compiler Features/Warnings

1. **-Wall (Enable all common warnings)**
Detects missing return values, uninitialized variables, unused variables, etc.
2. **-Wextra (Enable extra warnings)**
Reports additional potential issues such as suspicious code and missing arguments.
3. **Address Sanitizer (-fsanitize=address) / Undefined Behavior Sanitizer (-fsanitize=undefined)**
Detects array out-of-bounds access, invalid pointer usage, division by zero, and other runtime memory errors.

(Alternative acceptable answers):

- Static code analysis tools (e.g., clang-tidy)
- Compiler optimization warnings (-O2 with warnings)
- IDE code analysis and IntelliSense

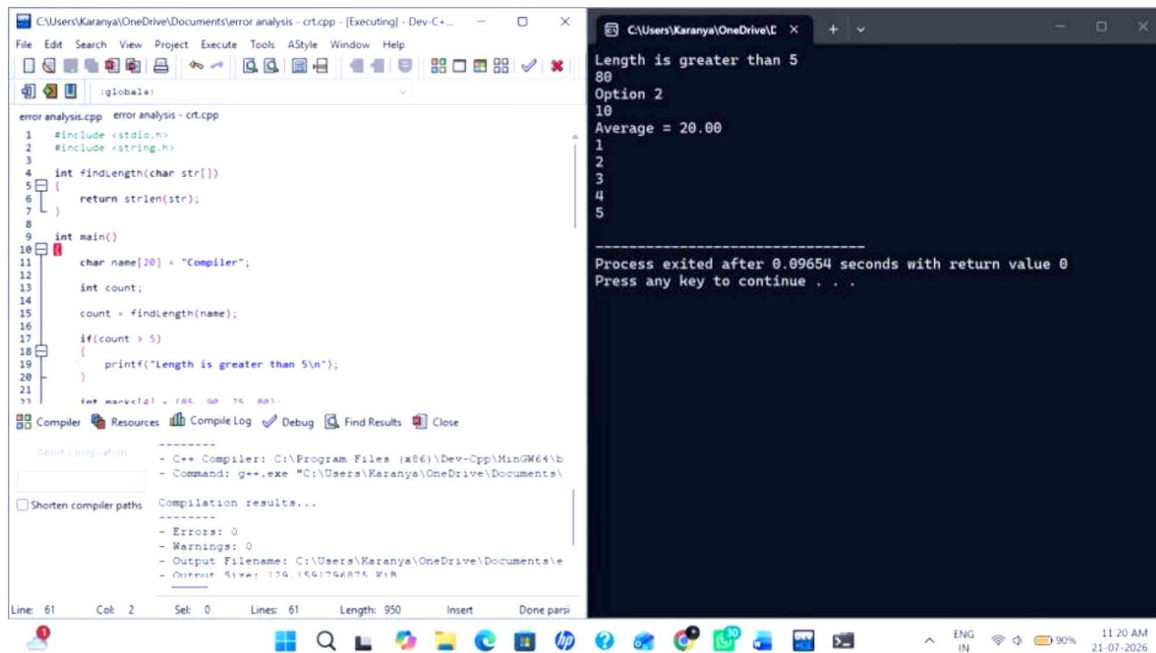
The screenshot shows the Dev-C++ IDE with a C++ file named `error analysis.cpp`. The code is as follows:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int findLength(char str[])
5 {
6     return strlen(str);
7 }
8
9 int main()
10 {
11     char name[20] = "Compiler"
12
13     int count;
14
15     count = findLength();
16
17     if(count > 5
18     {
19         printf("Length is greater than 5\n");
20     }
21 }
```

The IDE's Compiler window shows the following error messages:

- Line 13, Col 5: In function 'int main()': [Error] expected ';' or ':' before 'int'
- Line 15, Col 5: [Error] 'count' was not declared in this scope
- Line 15, Col 24: [Error] too few arguments to function: 'int findLength(char)'
- Line 4, Col 5: [Note] declared here
- Line 18, Col 5: [Error] expected ')' before '{' token
- Line 24, Col 20: [Error] 'marks' was not declared in this scope
- Line 35, Col 13: [Error] expected ')' before 'printf'

The status bar at the bottom indicates: Line: 13, Col: 5, Sel: 0, Lines: 60, Length: 854, Insert, Done parsing in 0.282 seconds. The system tray shows the date and time as 11:16 AM, 21-07-2026.



Q2 Solution

(a) Identify all syntax errors

Line No.	Syntax Error	Reason
3	<code>int maximum(int x, int y</code>	Missing closing parenthesis <code>)</code> in the function definition.
15	<code>printf("Maximum is 80\n")</code>	Missing semicolon <code>;</code> after <code>printf</code> .
17	<code>case 90</code>	Missing colon <code>:</code> after case 90.
24	<code>do loop</code>	Missing closing brace <code>}</code> before the <code>while(max > 75);</code> .
25	<code>while(max > 75);</code>	<code>while</code> should come after the closing brace of the <code>do</code> block.
27	<code>printf("Limit Reached\n"</code>	Missing closing parenthesis <code>)</code> and missing semicolon <code>;</code> .
29	End of <code>main()</code>	Missing closing brace <code>}</code> for <code>main()</code> function.

(b) Classification of Syntax Errors

Error	Category
Missing <code>)</code> in function definition	Unmatched Parentheses
Missing <code>;</code> after <code>printf</code>	Missing Delimiter (Semicolon)
Missing <code>:</code> after case 90	Malformed Switch Statement
Missing <code>}</code> before <code>while</code>	Unmatched Braces
Incorrect <code>do-while</code> structure	Malformed Control Structure

Missing) and ; in printf	Incomplete Statement
Missing closing } of main()	Unmatched Braces

(c) Corrected Program

```
#include <stdio.h>

int maximum(int x, int y)
{
    if(x > y)
        return x;
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n");
            break;

        case 90:
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }
}
```

```

do
{
    printf("%d\n", max);
    max--;
}
while(max > 75);

if(max == 75)
    printf("Limit Reached\n");

return 0;
}

```

(d) How the parser detects syntax errors and performs error recovery

Syntax Error Detection:

- The parser checks whether the program follows the grammar rules of the C language.
- It compares the sequence of tokens generated by the lexical analyzer with the grammar.
- If a token is missing, misplaced, or unexpected (such as a missing semicolon or parenthesis), the parser reports a syntax error.

Error Recovery:

- Instead of stopping immediately, the parser attempts to recover by skipping incorrect tokens or inserting expected tokens.
 - This allows the compiler to continue parsing the remaining program and report multiple syntax errors in a single compilation.
-

(e) Two improvements for better syntax error messages

1. **Provide precise error location**
 - Display the exact line number, column number, and the unexpected token to help programmers locate the error quickly.
2. **Suggest possible corrections**
 - Give hints such as:
 - "Did you forget a semicolon (;)?"
 - "Expected : after case."
 - "Missing closing parenthesis)."

These improvements make compiler error messages clearer and help programmers fix syntax errors more efficiently.

```

1 #include <stdio.h>
2
3 int maximum(int x, int y)
4 {
5     if(x > y)
6         return x;
7     else
8         return y;
9 }
10
11 int main()
12 {
13     int marks[5] = {70, 80, 90, 85, 95};
14     int max = maximum(marks[0], marks[1]);
15     switch(max)
16     {
17         case 80:
18             printf("Maximum is 80\n");
19             break;
20     }
21 }

```

Compiler (9) Resources Compile Log Debug Find Results Close

Line	Col	File	Message
4	1	C:\Users\Karanya\OneDrive\Documents\error analysis 2...	[Error] expected '}' before '{' token
21	13	C:\Users\Karanya\OneDrive\Documents\error analysis 2...	In function 'int main()':
24	13	C:\Users\Karanya\OneDrive\Documents\error analysis 2...	[Error] expected ';' before 'break'
39	5	C:\Users\Karanya\OneDrive\Documents\error analysis 2...	[Error] expected ';' before 'printf'
41	8	C:\Users\Karanya\OneDrive\Documents\error analysis 2...	[Error] expected ';' before '}' token
41	8	C:\Users\Karanya\OneDrive\Documents\error analysis 2...	[Error] expected 'while' before numeric constant
41	8	C:\Users\Karanya\OneDrive\Documents\error analysis 2...	[Error] expected ';' before numeric constant
41	9	C:\Users\Karanya\OneDrive\Documents\error analysis 2...	[Error] expected ';' before '}' token

Line: 4 Col: 1 Sel: 0 Lines: 47 Length: 641 Insert Done parsing in 0.015 seconds

```

20 printf("Maximum is 80\n");
21 break;
22
23 case 90:
24     printf("Maximum is 90\n");
25     break;
26
27 default:
28     printf("Other Value\n");
29 }
30
31 do
32 {
33     printf("%d\n", max);
34     max--;
35 } while(max > 75);
36
37 if(max == 75)
38     printf("Limit Reached\n");
39
40 return 0;
41

```

Compiler Resources Compile Log Debug Find Results Close

Compilation results...

- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\Karanya\OneDrive\Documents\error analysis 2- crt.c
- Output Size: 120.1416015625 KiB
- Compilation Time: 1.98s

Maximum is 80
80
79
78
77
76
Limit Reached

Process exited after 0.1225 seconds with return value 0
Press any key to continue . . .

Line: 41 Col: 2 Sel: 0 Lines: 41 Length: 636 Insert Done parsing

Q3 Solution

(a) Identify all errors in the program

Line No.	Error	Error Type	Reason
10	int @num = 5;	Lexical Error	@ is an invalid character in a C identifier.
11	int value = 10	Syntax Error	Missing semicolon (;).
13	result = factorial();	Semantic Error	factorial() requires one argument, but none is provided.

23	case 120	Syntax Error	Missing colon (:) after case 120.
30	printf("%d\n", numbers[5]);	Runtime Error	Array index out of bounds. Valid indices are 0–4.
32	printf("%d\n", *ptr);	Runtime Error	Dereferencing an uninitialized pointer.
35	int c = a / b;	Runtime Error	Division by zero (b = 0).
36	int sum = result + 0 + value;	Code Optimization	Adding 0 is redundant and can be optimized to result + value.
38–41	while(i <= 5)	Logical / Runtime Error	i is never incremented, causing an infinite loop.

(b) Classification according to compiler phases

Compiler Phase	Error
Lexical Analysis	Invalid identifier @num.
Syntax Analysis	Missing semicolon after int value = 10; Missing : after case 120.
Semantic Analysis	Function call factorial() without the required argument.
Intermediate Code Generation	No direct error; intermediate code cannot be generated correctly until syntax and semantic errors are fixed.
Code Optimization	Redundant expression result + 0 + value; the + 0 can be removed.
Runtime	Array out-of-bounds (numbers[5]), uninitialized pointer dereference, division by zero, infinite loop.

(c) Compiler processing and error detection

Phase	Errors Detected
Lexical Analysis	Detects invalid tokens such as @ in identifiers.
Syntax Analysis (Parser)	Detects missing semicolons, colons, parentheses, braces, and other grammar violations.
Semantic Analysis	Checks function arguments, variable declarations, and type correctness. Detects the incorrect call to factorial().
Intermediate Code Generation	Generates intermediate representation only if lexical, syntax, and semantic errors are resolved.
Code Optimization	Simplifies redundant expressions such as result + 0 + value to result + value.
Runtime (Execution)	Detects array index out of bounds, division by zero, invalid pointer dereference, and infinite loops during execution.

(d) Corrected Program

```
#include <stdio.h>
```

```

int factorial(int n)
{
    if(n <= 1)
        return 1;
    return n * factorial(n - 1);
}

int main()
{
    int num = 5;
    int value = 10;
    int result;

    result = factorial(num);

    if(result >= 100)
    {
        printf("Large Number\n");
    }

    switch(result)
    {
        case 24:
            printf("Factorial of 4\n");
            break;

        case 120:
            printf("Factorial of 5\n");
            break;

        default:
            printf("Other Value\n");
    }
}

```

```

int numbers[5] = {2, 4, 6, 8, 10};
printf("%d\n", numbers[4]);

int data = 50;
int *ptr = &data;
printf("%d\n", *ptr);

int a = 50;
int b = 5;
int c = a / b;
printf("%d\n", c);

int sum = result + value;
printf("Sum = %d\n", sum);

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
    i++;
}

return 0;
}

```

(e) Two compiler improvements

1. **Enable advanced compiler warnings (-Wall -Wextra)**
 These warnings help detect issues such as missing function arguments, uninitialized variables, unreachable code, and other programming mistakes before execution.
2. **Use static analysis and optimization tools**
 Static analyzers (e.g., Clang Static Analyzer, clang-tidy) and optimization passes can identify redundant expressions, possible division by zero, out-of-bounds array access, and potential null or uninitialized pointer dereferences before the program runs.

C:\Users\Karanya\OneDrive\Documents\error analysis 3.cpp - Dev-C++ 5.11

```

1  #include <stdio.h>
2
3  int factorial(int n)
4  {
5      if(n <= 1)
6          return 1;
7      return n * factorial(n - 1);
8  }
9
10 int main()
11 {
12     int num = 5;
13     int value = 10;
14     int result;
15     result = factorial();
16     if(result >= 100)
17     {
18         printf("Large Number\n");
19     }
20 }

```

Compiler (7) Resources Compile Log Debug Find Results Close

Line	Col	File	Message
13	9	C:\Users\Karanya\OneDrive\Documents\error analysis 3.cpp	[Error] stray '@' in program
17	5	C:\Users\Karanya\OneDrive\Documents\error analysis 3.cpp	In function 'int main()': [Error] expected ';' or '.' before 'int'
19	5	C:\Users\Karanya\OneDrive\Documents\error analysis 3.cpp	[Error] result was not declared in this scope
19	24	C:\Users\Karanya\OneDrive\Documents\error analysis 3.cpp	[Error] too few arguments to function 'int factorial(int)'
3	5	C:\Users\Karanya\OneDrive\Documents\error analysis 3.cpp	[Note] declared here
33	13	C:\Users\Karanya\OneDrive\Documents\error analysis 3.cpp	[Error] expected ';' before 'printf'

Line: 13 Col: 9 Sel: 0 Lines: 61 Length: 865 Insert Done parsing in 0.016 seconds

C:\Users\Karanya\OneDrive\Documents\error analysis 3.cpp - [Executing] - Dev-C++ 5.11

```

1  #include <stdio.h>
2
3  int factorial(int n)
4  {
5      if(n <= 1)
6          return 1;
7      return n * factorial(n - 1);
8  }
9
10 int main()
11 {
12     int num = 5;
13     int value = 10;
14     int result;
15     result = factorial(num);
16 }

```

Compiler (7) Resources Compile Log Debug Find Results Close

Compilation results...

- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\Karanya\OneDrive\Documents\error analysis 3.exe
- Output Size: 128.65625 KiB
- Compilation Time: 0.33s

Line: 63 Col: 2 Sel: 0 Lines: 63 Length: 948 Insert Done parsing

Large Number
Factorial of 5
10
100
10
130
1
2
3
4
5

Process exited after 0.1124 seconds with return value 0
Press any key to continue . . .

Tasks

(a) Identify all the errors in the program. Mention the line number, error type, and reason for each error. (4 Marks)

(b) Classify the identified errors according to the appropriate compiler phase (Lexical, Syntax, Semantic, Intermediate Code Generation, Code Optimization, and Runtime) using a suitable table. (4 Marks)

(c) Explain how the compiler processes the program by identifying the errors detected during each compilation phase and those detected only at runtime. (3 Marks)

(d) Rewrite the complete corrected program so that it compiles and executes without errors. (2 Marks)

(e) Suggest any two improvements in compiler design or optimization techniques that could detect or prevent such errors before program execution. (2 Marks)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand